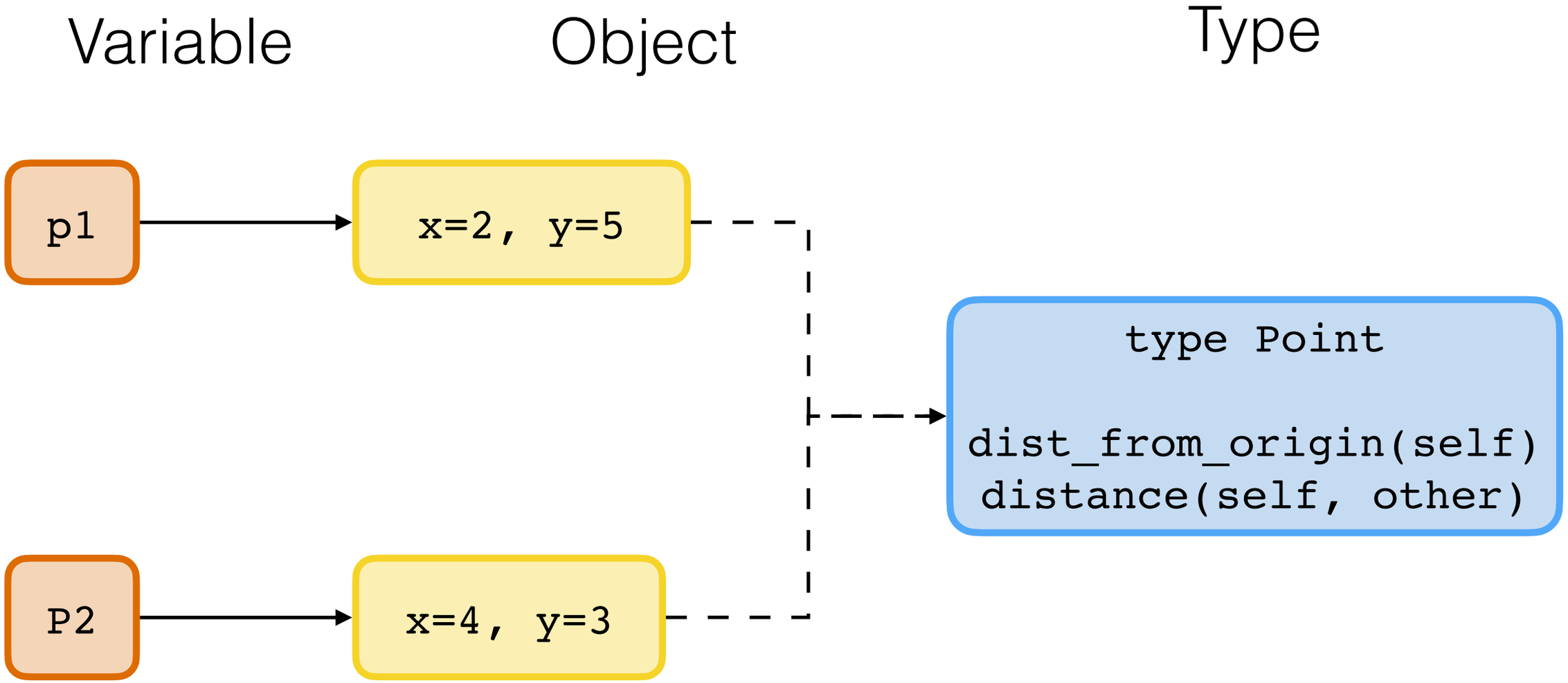
Modules
(Using code from other files)

Summary

- You can **`import`** definitions from other files.
- Imported files are called modules.
- Modules may be your own files with code or a python library you installed with pixi, conda, pip, or uv.
- Project on finding open reading frames.

A `Point` class



```
class Point(object):
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __str__(self):
        return "({}, {})".format(self.x, self.y)

    def __add__(self, other):
        new_x = self.x + other.x
        new_y = self.y + other.y
        return Point(new_x, new_y)

def __eq__(self, other):
    return self.x == other.x and self.y == other.y

def dist_from_origin(self):
    return (self.x ** 2 + self.y ** 2) ** 0.5

def distance(self, other):
    a = self.x - other.x
    b = self.y - other.y
    return (a ** 2 + b ** 2) ** 0.5
```

```
p1 = Point(4, 2)
p2 = Point(3, 7)

new_p = p1 + p2
```

```
force = p1.dist_from_origin()
# Point.dist_from_origin(p1)

dist = p1.distance(p2)
# Point.distance(p1, p2)
```

```python
class Point(object):
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __str__(self):
        return "({}, {})".format(self.x, self.y)

    def __add__(self, other):
        new_x = self.x + other.x
        new_y = self.y + other.y
        return Point(new_x, new_y)
```

point.py

```python
    def __eq__(self, other):
        return self.x == other.x and self.y == other.y

    def dist_from_origin(self):
        return (self.x ** 2 + self.y ** 2) ** 0.5

    def distance(self, other):
        a = self.x - other.x
        b = self.y - other.y
        return (a ** 2 + b ** 2) ** 0.5
```

import

```
import point
```

- Finds **`point.py`** running through a predefined list of folders.

- Runs **`point.py`** (just like **`python point.py`**)

- Creates a **`module`** object that hold the resulting definitions and assigns that to the name **`point`**

import

```
import point

p1 = point.Point(4, 2)
p2 = point.Point(3, 7)

dist = p1.distance(p2)
```

import

```
from point import Point

p1 = Point(4, 2)
p2 = Point(3, 7)

dist = p1.distance(p2)
```

seqtools.py

```
def gc_content(dna):
    "Fraction of the bases that are G or C."
    gc = dna.count('G') + dna.count('C')
    return gc / len(dna)
```

analyze.py

```
import seqtools

gc = seqtools.gc_content('ATGC')
print(gc)
```

```
python analyze.py
```

Python Tools for Computational Molecular Biology

Documentation
Download
Mailing lists
News
Biopython Contributors
Scriptcentral
Source Code
GitHub project

Biopython version 1.72
© 2018. All rights reserved.

Biopython

See also our News feed and Twitter.

Introduction

Biopython is a set of freely available tools for biological computation written in Python by an international team of developers.

It is a distributed collaborative effort to develop Python libraries and applications which address the needs of current and future work in bioinformatics. The source code is made available under the Biopython License, which is extremely liberal and compatible with almost every license in the world.

We are a member project of the Open Bioinformatics Foundation (OBF), who take care of our domain name and hosting for our mailing list etc. The OBF used to host our developement repository, issue tracker and website but these are now on GitHub.

This wiki will help you download and install Biopython, and start using the libraries and tools.

Get Started	Get help	Contribute
Download Biopython	Tutorial (PDF)	What's being worked on
Installation help (PDF)	Documentation on this wiki	Developing on Github
	Cookbook (working examples)	Google Summer of Code
	Discuss and ask questions	Report bugs

The latest release is Biopython 1.72, released on 26 June 2018.

Install

Getting Started

Documentation

Report Bugs

Blogs

SciPy (pronounced “Sigh Pie”) is a Python-based ecosystem of open-source software for mathematics, science, and engineering. In particular, these are some of the core packages:

NumPy
Base N-dimensional array package

SciPy library
Fundamental library for scientific computing

Matplotlib
Comprehensive 2D Plotting

IPython
Enhanced Interactive Console

Sympy
Symbolic mathematics

pandas
Data structures & analysis

More information...

News

NumPy 1.15.4 released (2018-11-04) See Obtaining NumPy & SciPy libraries.

NumPy 1.15.3 released (2018-10-22) See Obtaining NumPy & SciPy libraries.

NumPy 1.14.6 released (2018-09-23) See Obtaining NumPy & SciPy libraries.

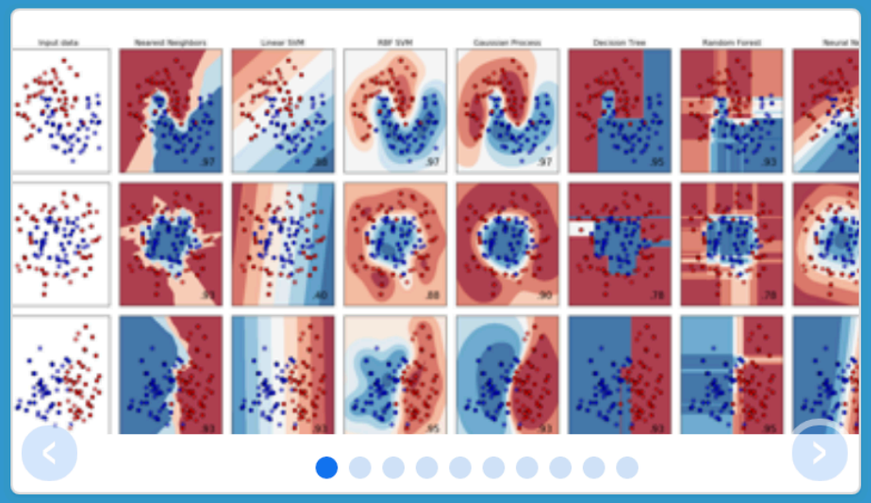
NumPy 1.15.2 released See Obtaining NumPy & SciPy libraries.

About SciPy
Getting Started
Documentation
Install
Bug Reports
Codes of Conduct
SciPy Conferences
Topical Software
Citing
Cookbook
Blogs
NumFOCUS

CORE PACKAGES:
Numpy
SciPy library
Matplotlib
IPython
Sympy
Pandas

Search

Go



scikit-learn

Machine Learning in Python

- Simple and efficient tools for data mining and data analysis
- Accessible to everybody, and reusable in various contexts
- Built on NumPy, SciPy, and matplotlib
- Open source, commercially usable - BSD license

Classification

Identifying to which category an object belongs to.

Applications: Spam detection, Image recognition.
Algorithms: SVM, nearest neighbors, random forest, …

— Examples

Regression

Predicting a continuous-valued attribute associated with an object.

Applications: Drug response, Stock prices.
Algorithms: SVR, ridge regression, Lasso, …

— Examples

Clustering

Automatic grouping of similar objects into sets.

Applications: Customer segmentation, Grouping experiment outcomes
Algorithms: k-Means, spectral clustering, mean-shift, …

— Examples

Dimensionality reduction

Reducing the number of random variables to consider.

Applications: Visualization, Increased efficiency
Algorithms: PCA, feature selection, non-negative matrix factorization.

— Examples

Model selection

Comparing, validating and choosing parameters and models.

Goal: Improved accuracy via parameter tuning
Modules: grid search, cross validation, metrics.

— Examples

Preprocessing

Feature extraction and normalization.

Application: Transforming input data such as text for use with machine learning algorithms.
Modules: preprocessing, feature extraction.

— Examples

News

On-going development: What's new (Changelog)

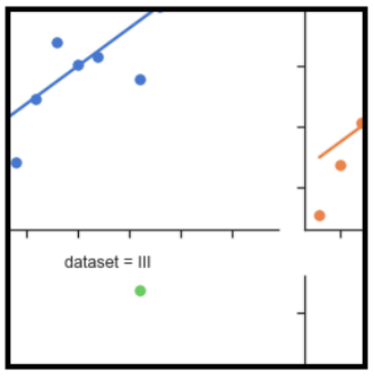
Community

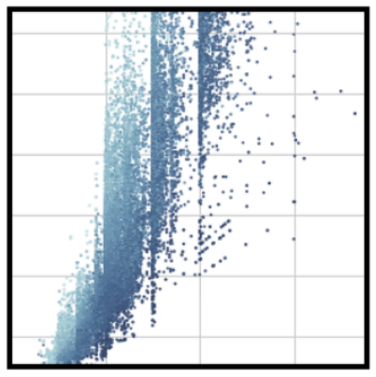
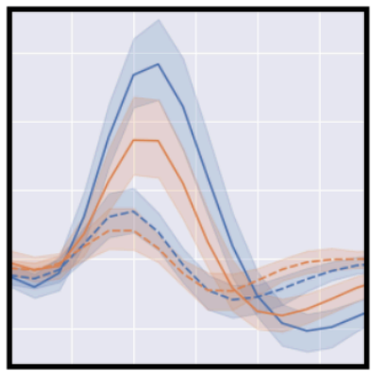
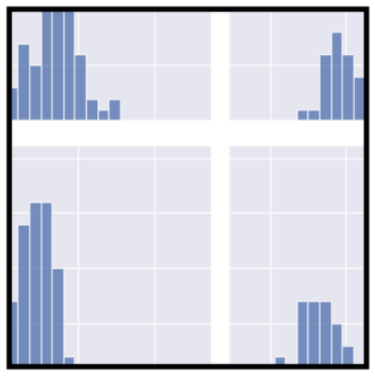
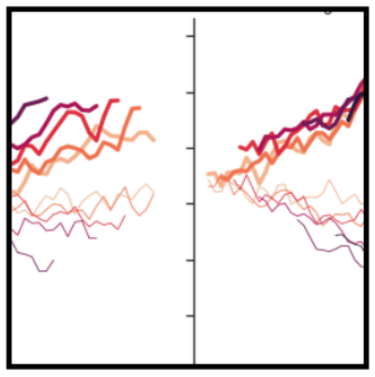
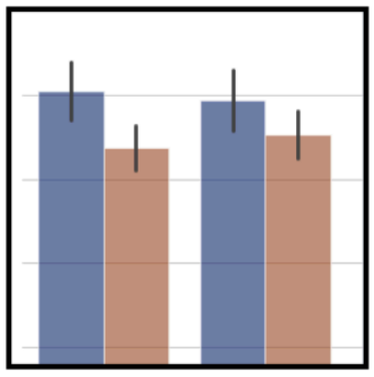
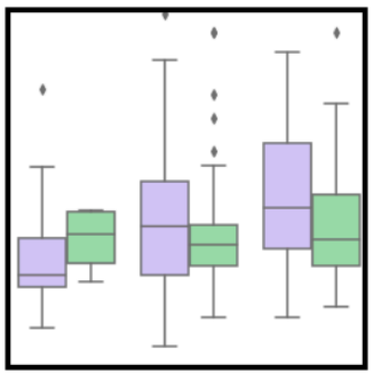
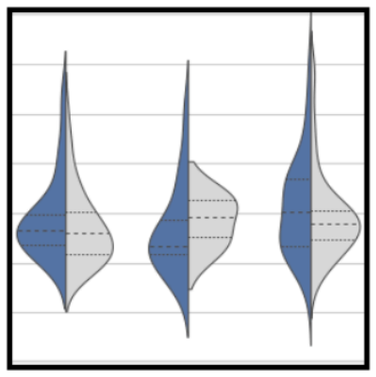
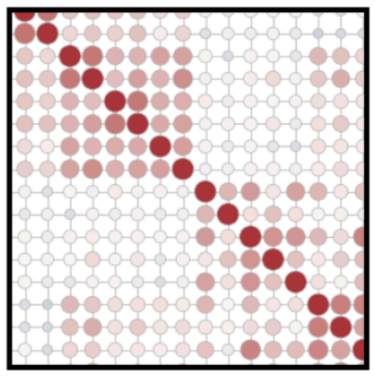
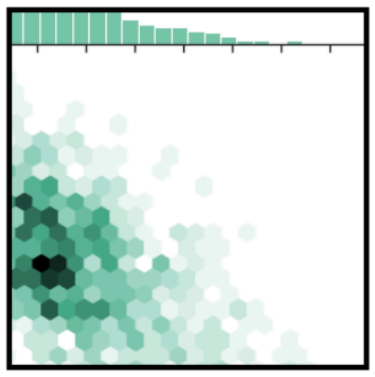
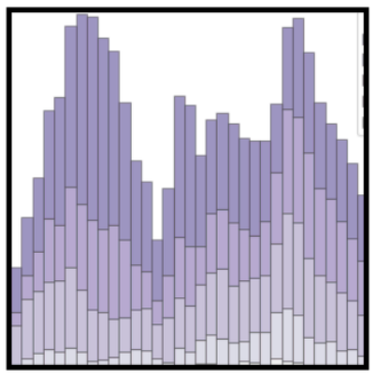
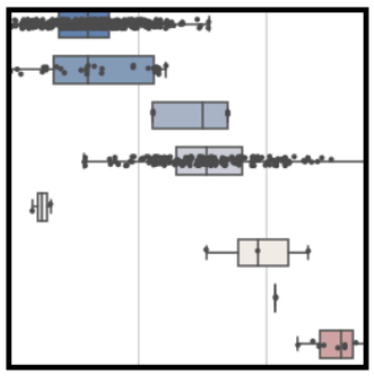
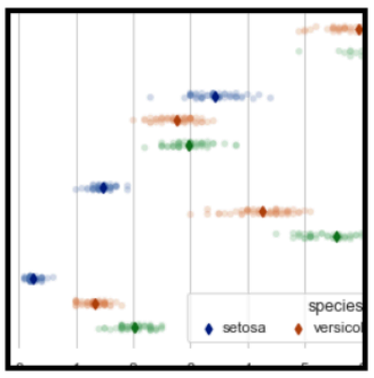
About us See authors and contributing

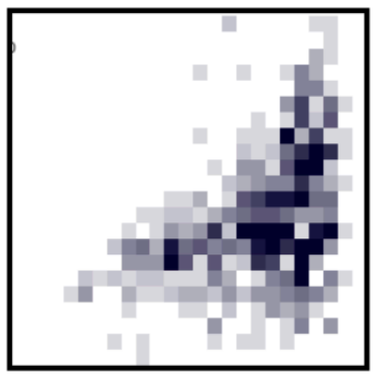
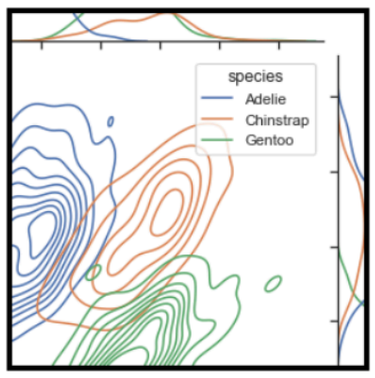
More Machine Learning Find related projects

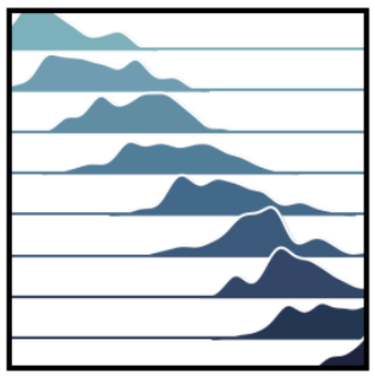
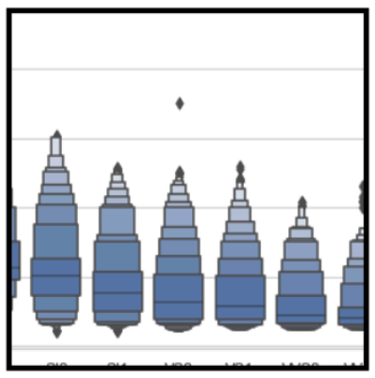
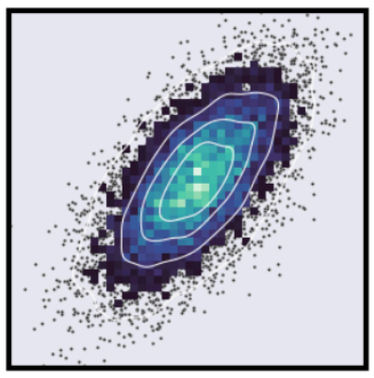
Who uses scikit-learn?

Example gallery



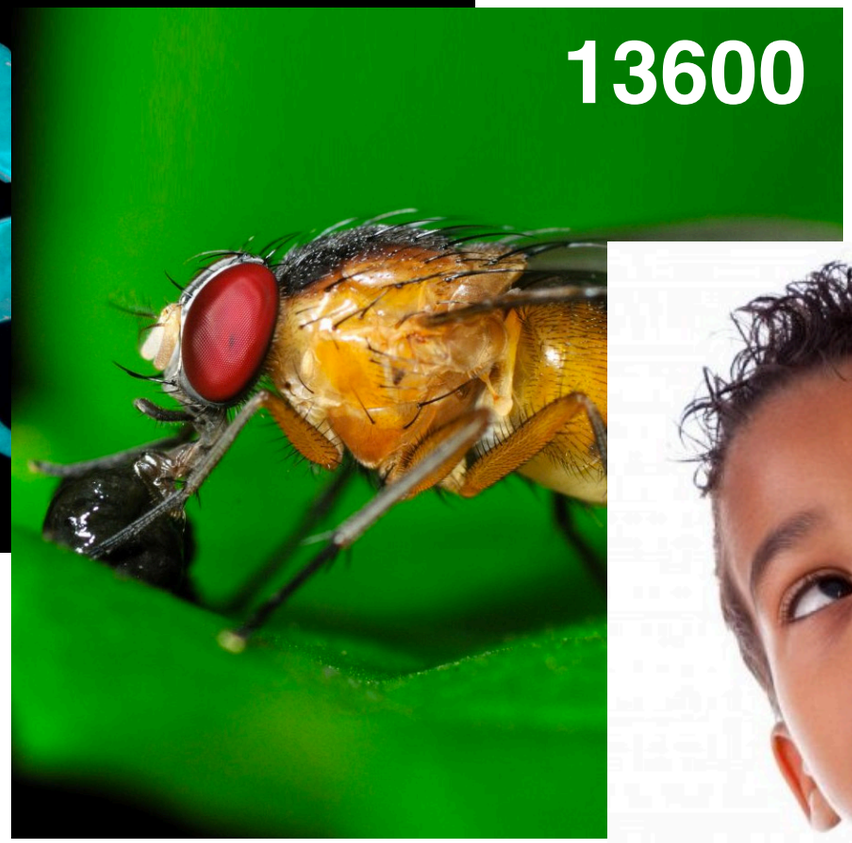













Project
Predicting proteins
in E. coli O157:H7

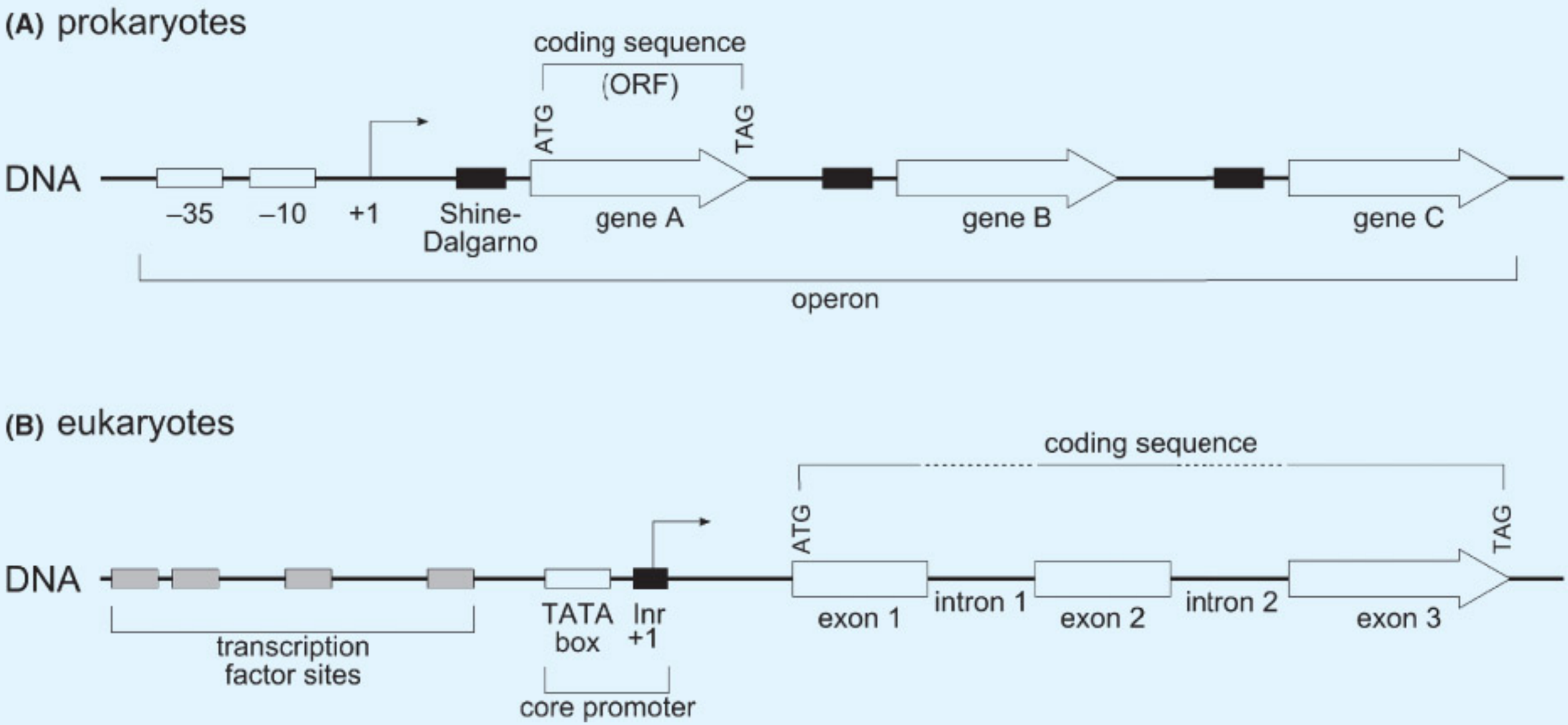
Genes in the genome



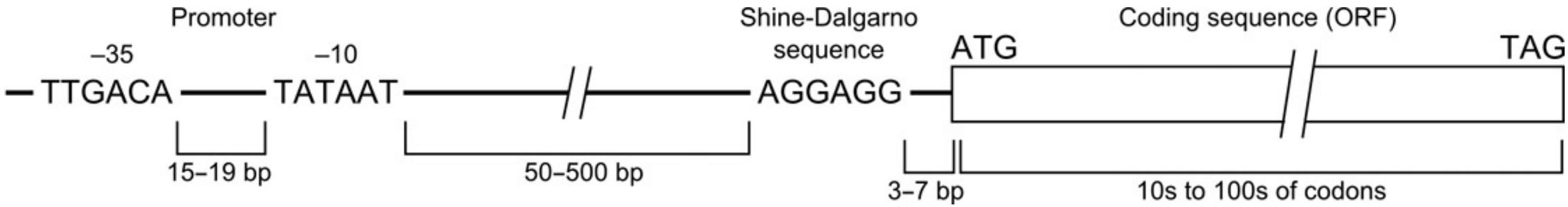
Genes and genomes

Organisme	Navn	Antal gener	Genom størrelse i Mb
Zea mays	**Majs**	**39656**	2300
Picea abies	**Rødgran**	**28354**	20000
Arabidopsis thaliana	**Alm. gåsemad**	**27655**	119
Caenorhabditis elegans	**Nematode**	**20470**	100
Homo sapiens	**Menneske**	**19989**	3200
Anopheles gambiae	**Myg (malaria vektor)**	**13683**	278
Drosophila melanogaster	**Bananflue**	**13600**	165
Dictyostelium discoideum	**Slimål**	**12500**	34
Saccharomyces cerevisiae	**Bagergær**	**6294**	121
E. coli O157:H7	**ond Coli bakterie**	**5416**	5.4
Plasmodium falciparum	**Protozo (malaria)**	**5268**	22.9
E. coli K-12	**Coli bakterie**	**4377**	4,6

Gene structure



Algorithm for “ORF finding” i bacteria?



De novo gene prediction

ORF finding

ATACAATGCACGCAATTGACTAGCA

?

ORF finding

ATACAATGCACGCAATTGACTAGCA

ORF finding

ATACAATGCACGCAATTGACTAGCA

ORF finding

ATACAATGCACGCAATTGACTAGCA

ORF finding

ATACAATGCACGCAATTGACTAGCA

ORF finding

ATACAATGCACGCAATTGACTAGCA

ORF finding

ATACAATGCACGCAATTGACTAGCA

ORF finding

ATACAATGCACGCAATTGACTAGCA

ORF finding

ATACAATGCACGCAATTGACTAGCA

ORF finding

ATACAATGCACGCAATTGACTAGCA

ORF finding

ATACAATGCACGCAATTGACTAGCA

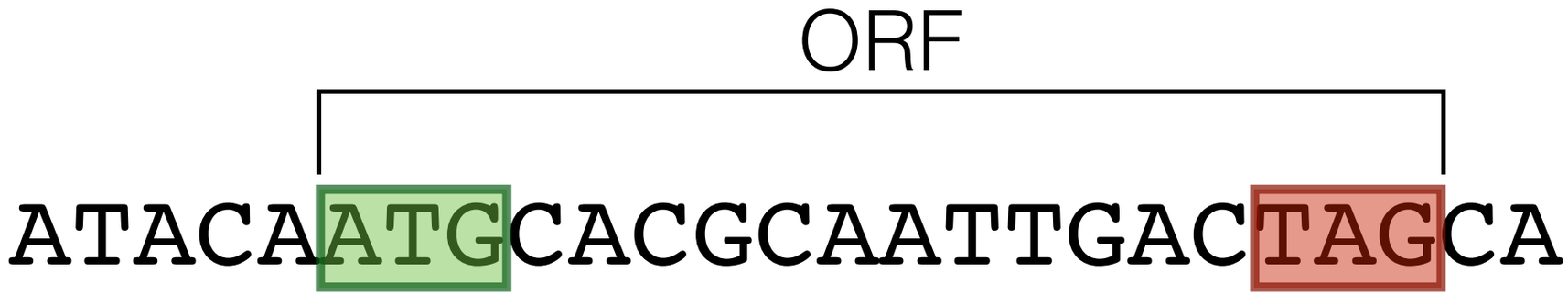
ORF finding

ATACAATGCACGCAATTGACTAGCA

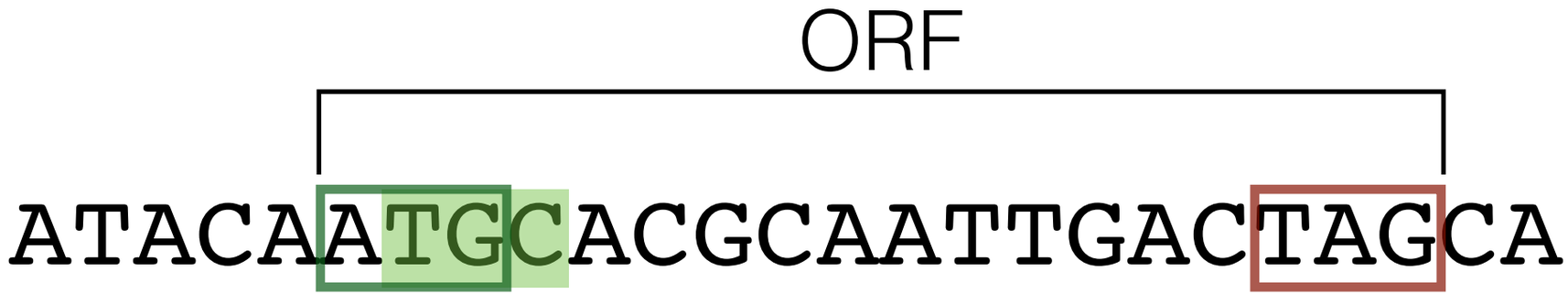
ORF finding

ATACAATGCACGCAATTGACTAGCA

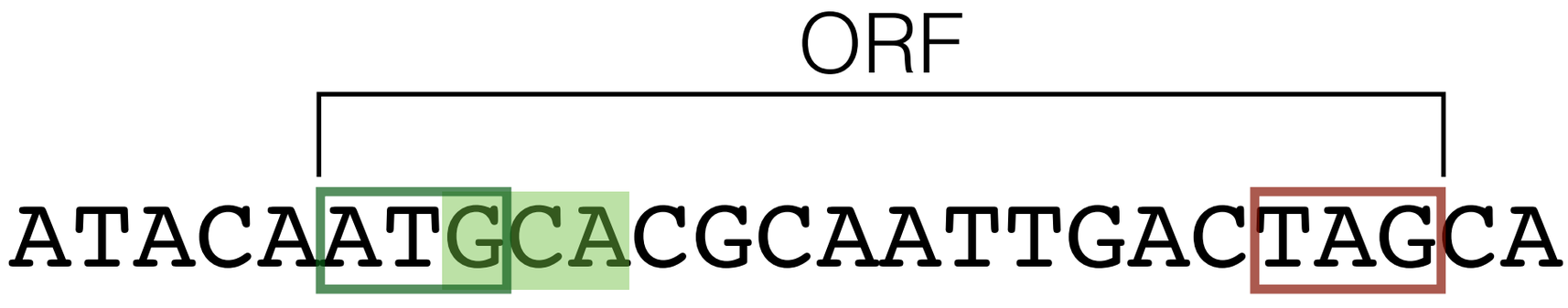
ORF finding



ORF finding

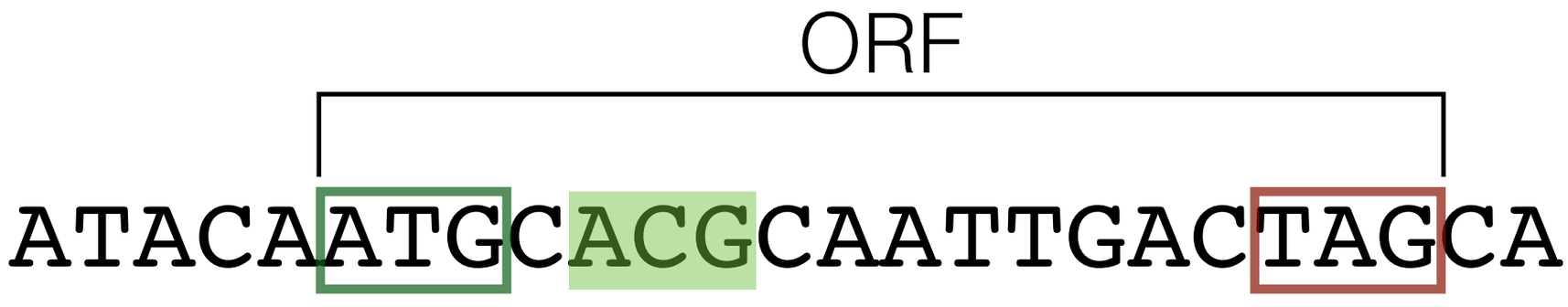


ORF finding

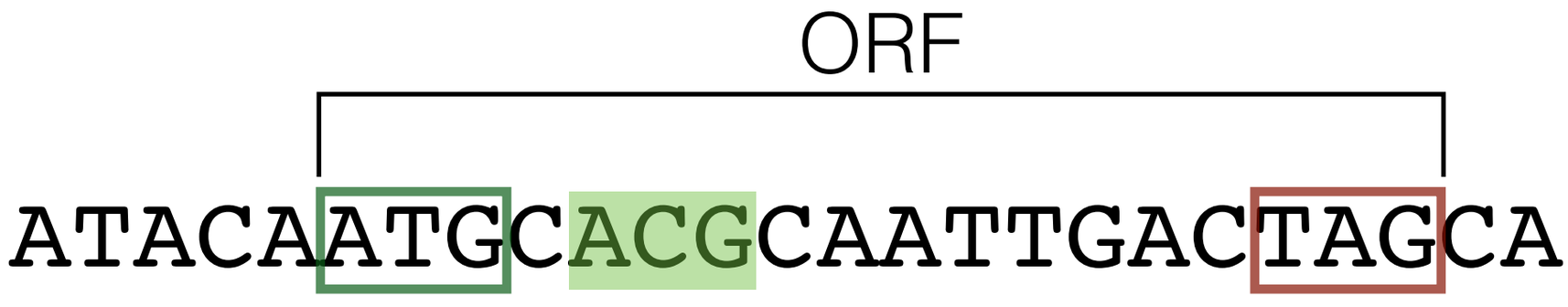


ORF finding

ORF finding

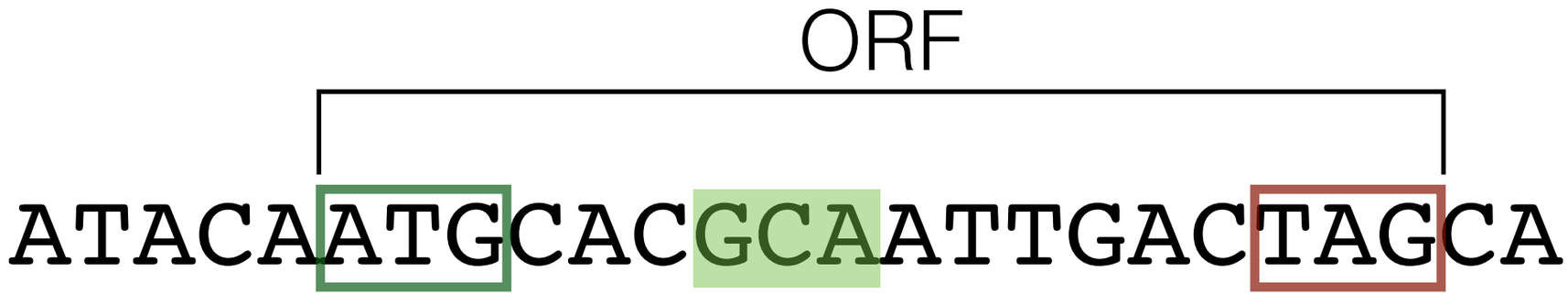


ORF finding

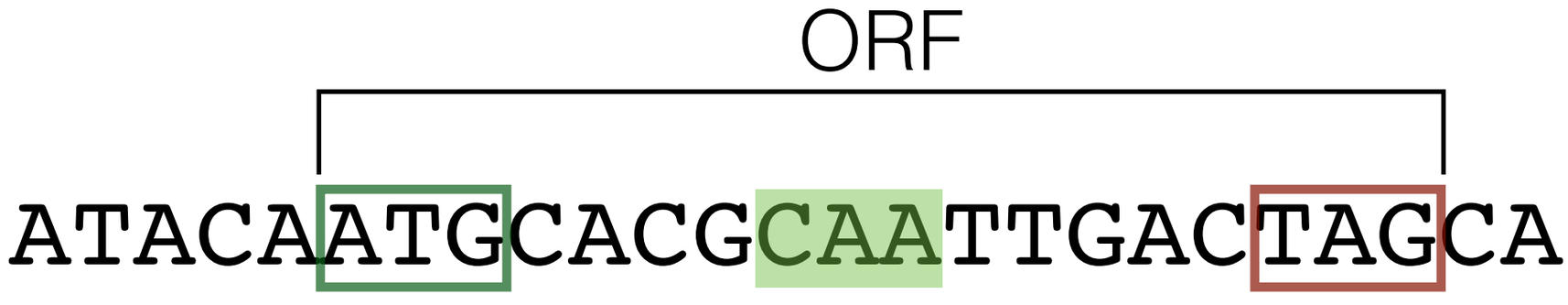


ORF finding

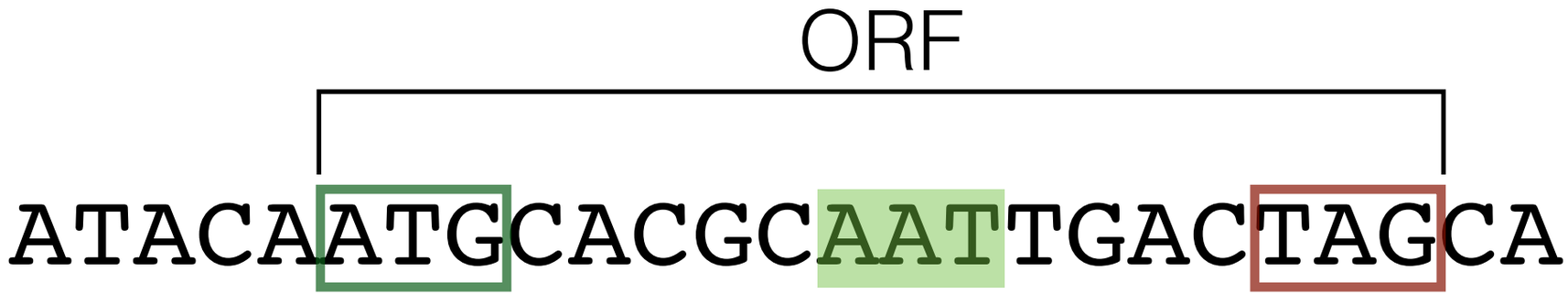
ORF finding



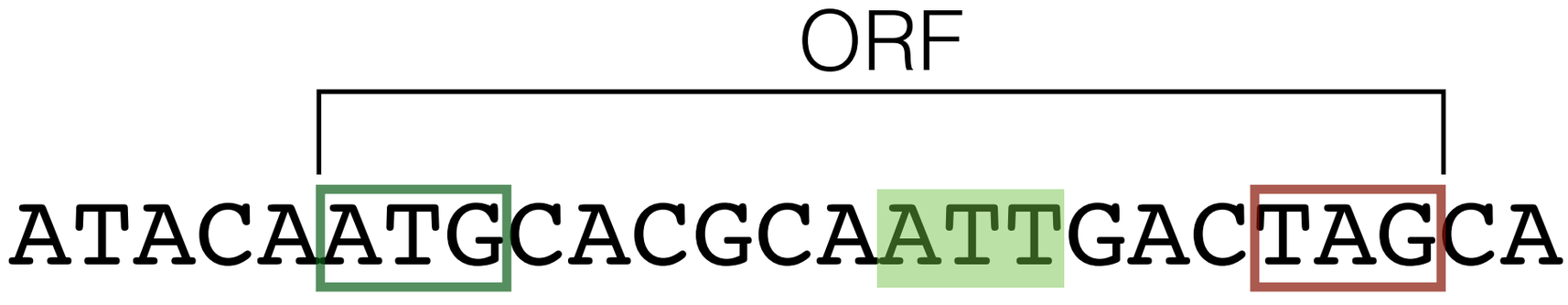
ORF finding



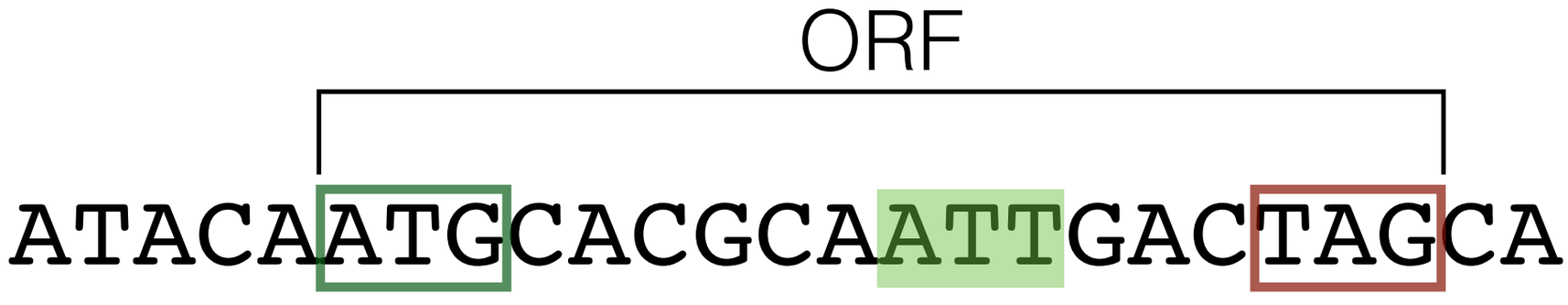
ORF finding



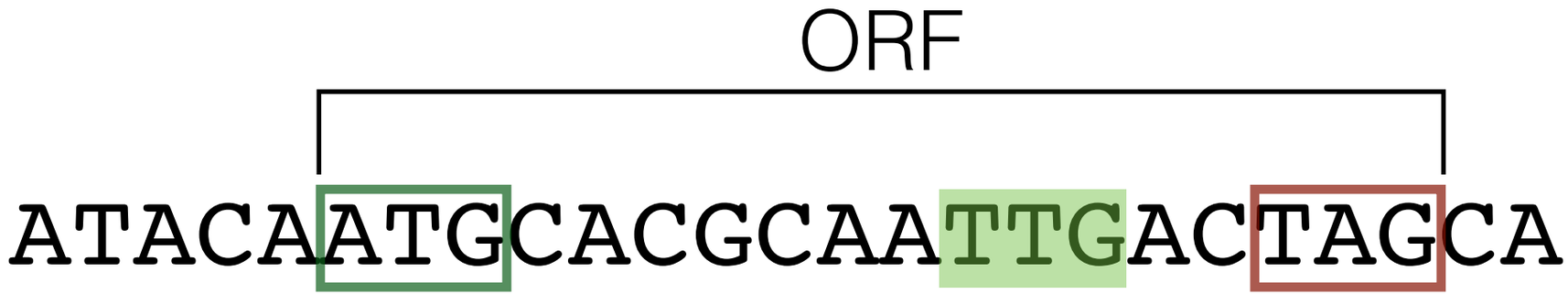
ORF finding



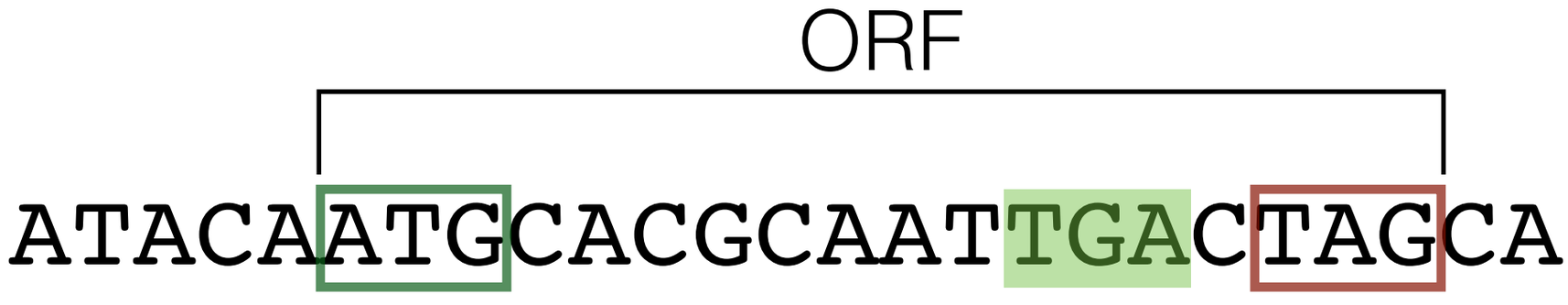
ORF finding



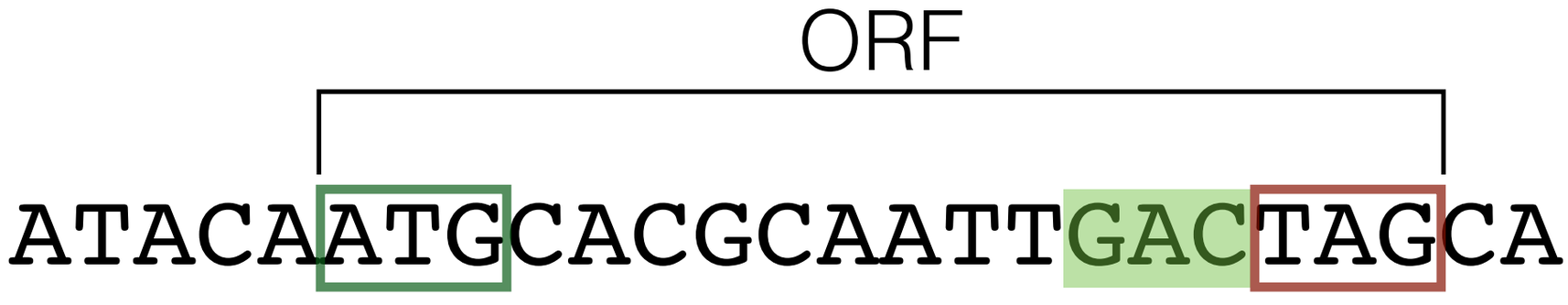
ORF finding



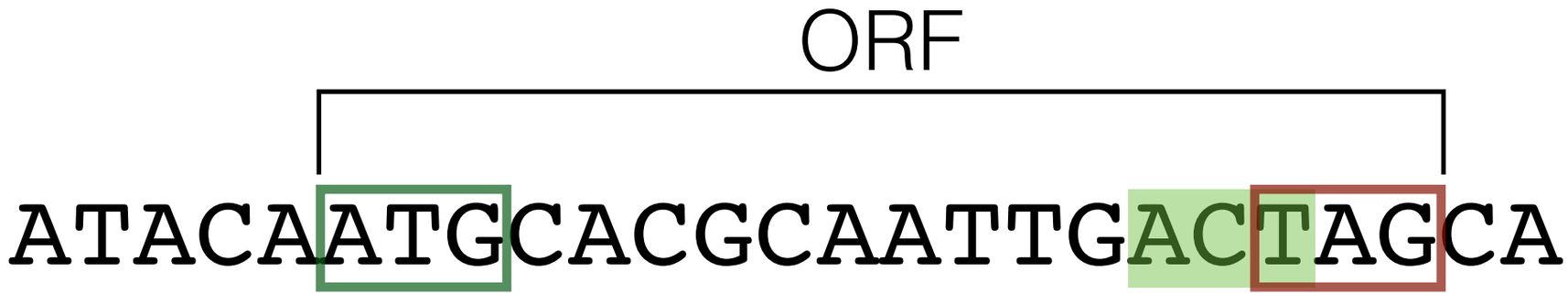
ORF finding



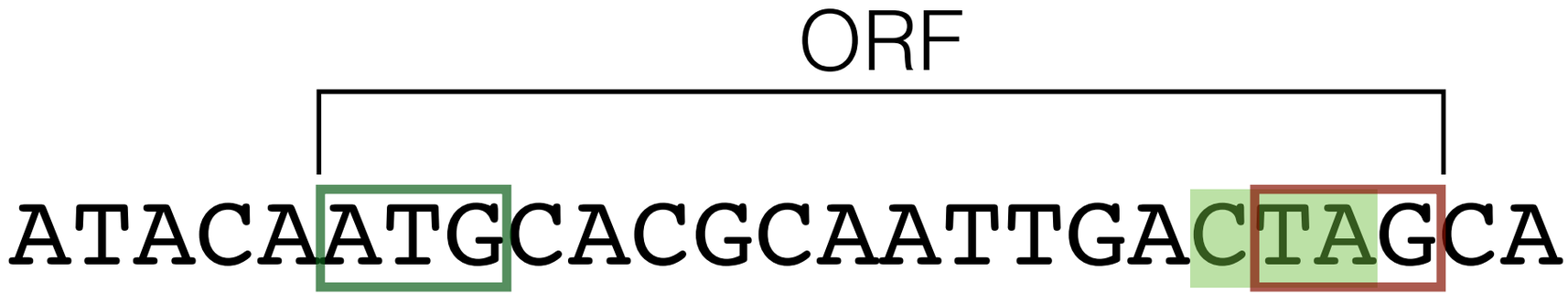
ORF finding



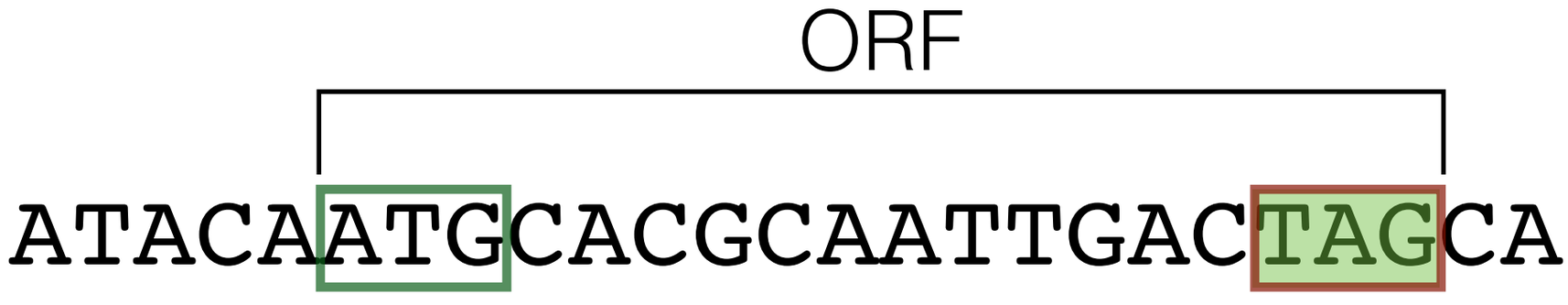
ORF finding



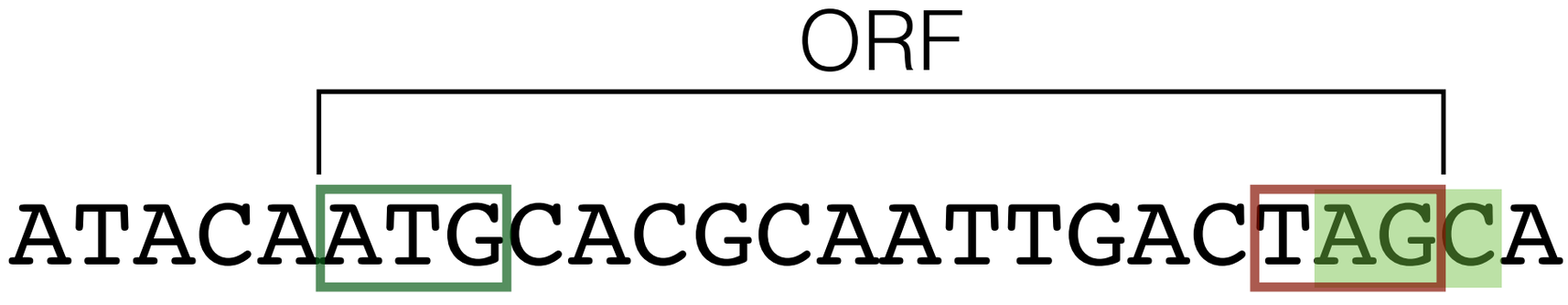
ORF finding



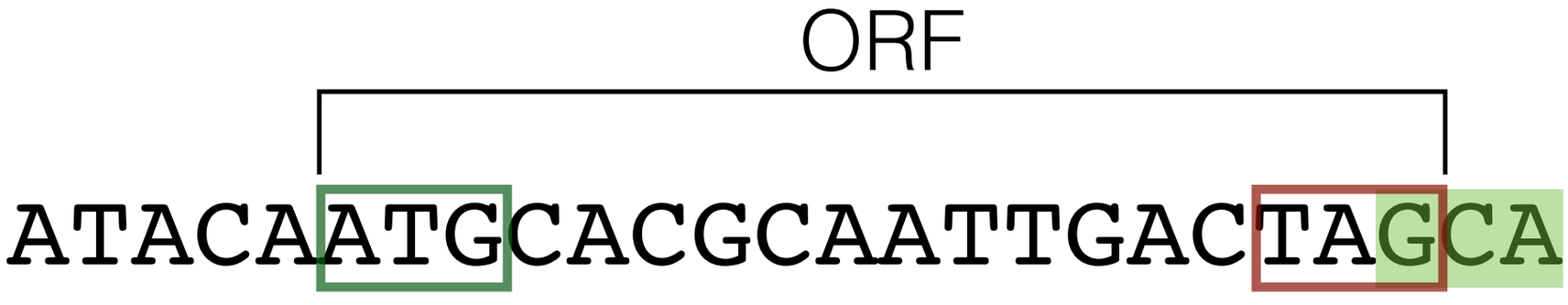
ORF finding



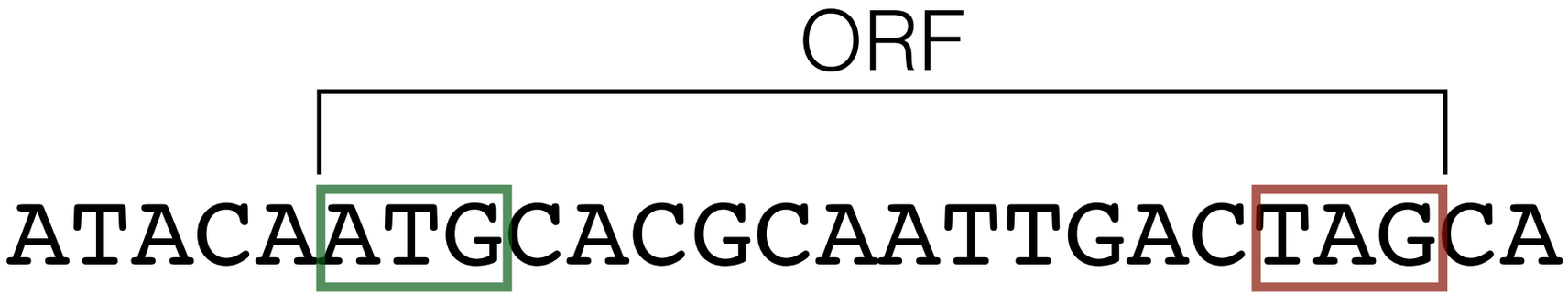
ORF finding



ORF finding



ORF finding



ORF finding

- Can our algorithm find:
  - ORFs in more than one reading frame?
  - ORFs that overlap?
  - ORFs on both DNA strings?

ORF finding

- How can we judge whether the ORFs we find are random or represent actual coding regions?

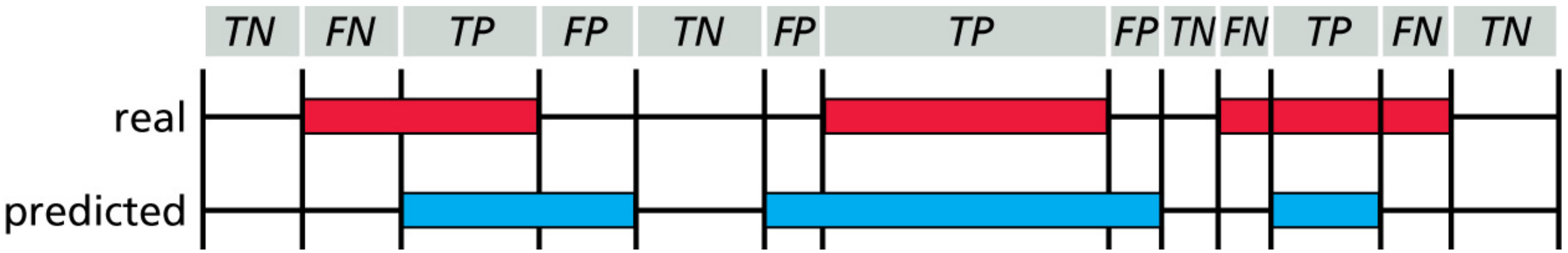
- Can we filter them in a way that makes the remaining ones more likely to be coding regions?

- Can we extend our altorithm to make it more specific?

Sensitivity and specificity

$$\text{Sensitivity} = \frac{\text{True ORF sequence found}}{\text{All the true ORF sequence}}$$

$$\text{Specificity} = \frac{\text{True ORF sequence found}}{\text{All ORF sequence found}}$$



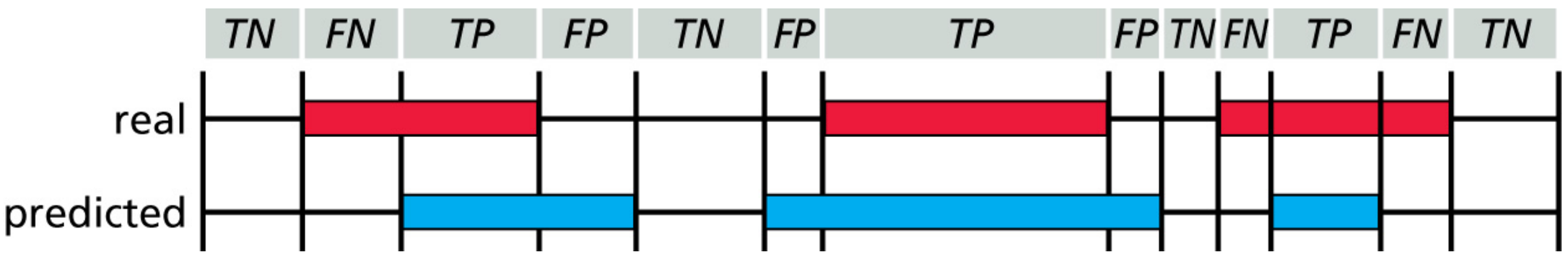
Sensitivity and specificity

TP: True positive (True ORF sequence found)

FP: False positive (True ORF sequence not found)

TN: True negative (False ORF sequence not found)

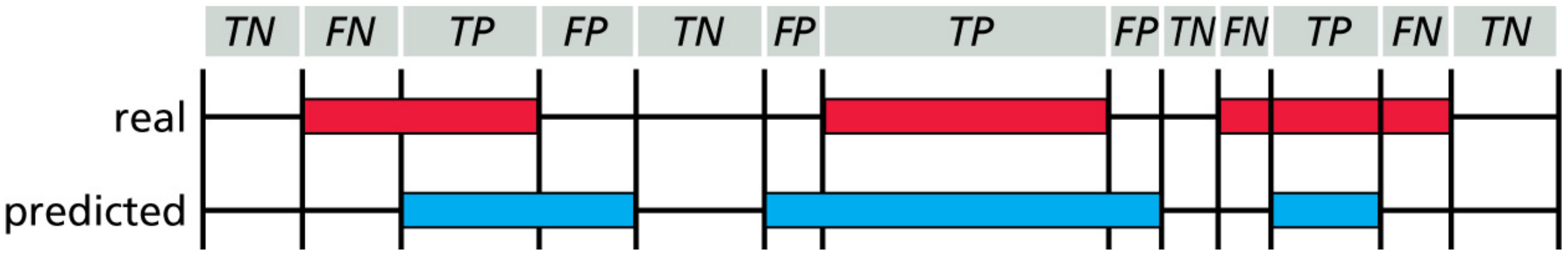
FN: False native (False ORF sequence found)



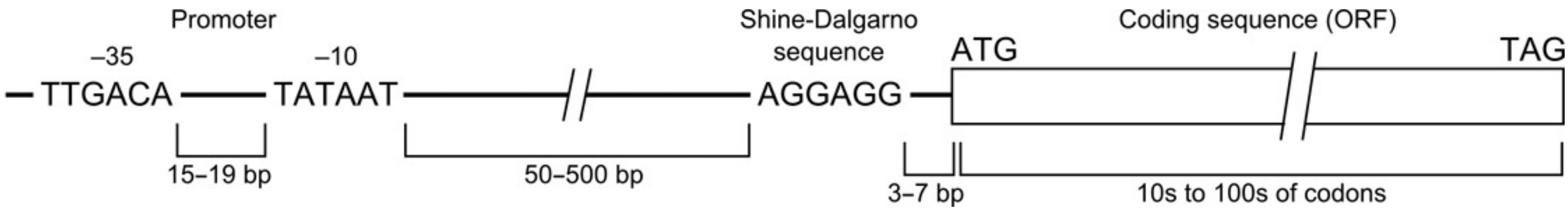
Sensitivity and specificity

$$\text{Sensitivity} = \frac{\text{True ORF sequence found}}{\text{All the true ORF sequence}} = \frac{TP}{TP + FN}$$

$$\text{Specificity} = \frac{\text{True ORF sequence found}}{\text{All ORF sequence found}} = \frac{TP}{TP + FP}$$



Sekvensmotiver



Motiv finding

Motiv: `TATAAT`

Sekvens: `GCACGATATAATCCTAGCAAATTGA`

?

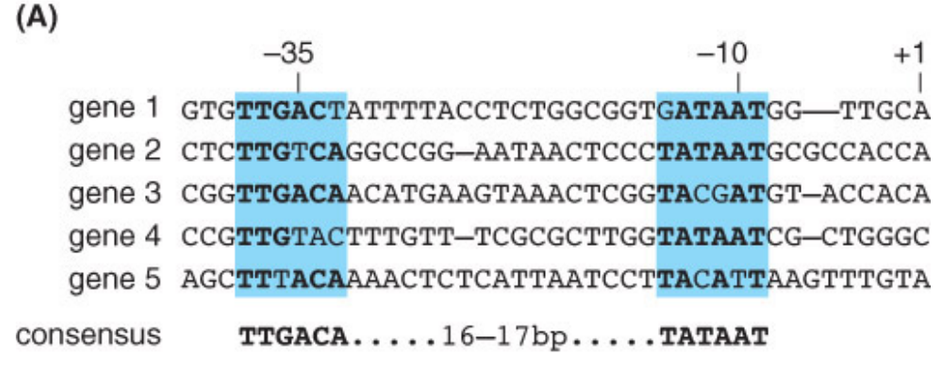
Motiv finding

-10 promoter consensus

TATAAT
GCACGATATAATCCTAGCAAATTGA

Motiv finding

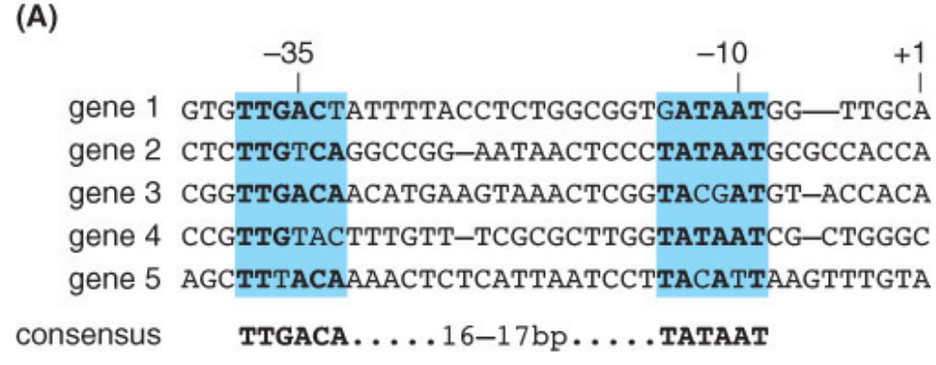
-10 promoter consensus



TATAAT
GCACGATATAATCCTAGCAAATTGA

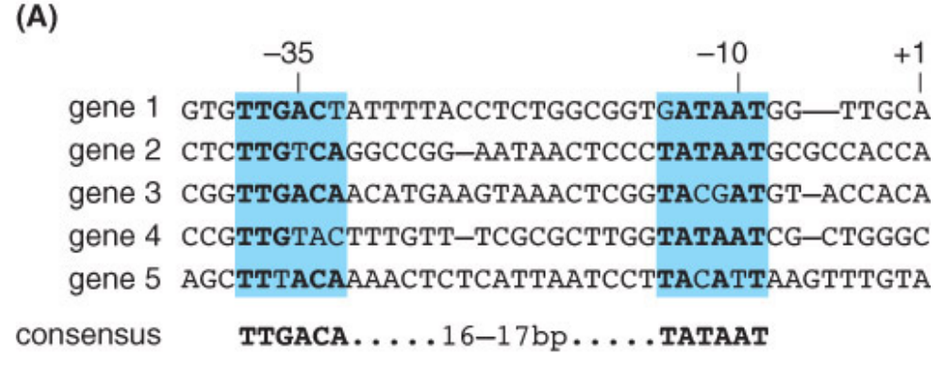
Motiv finding

-10 promoter consensus



Motiv finding

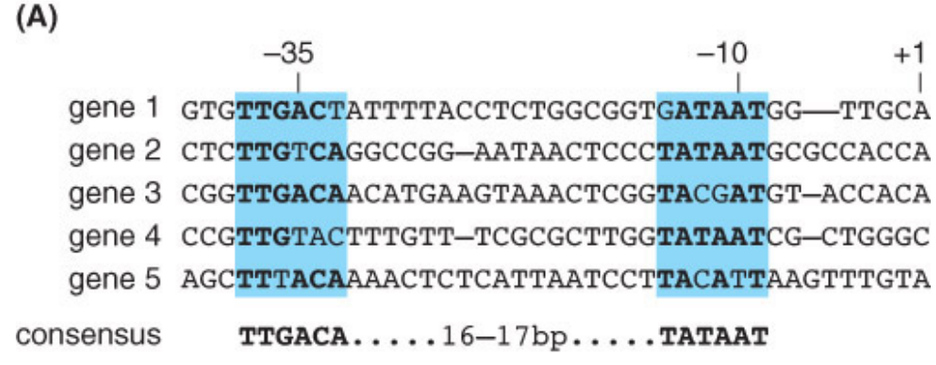
-10 promoter consensus



TATAAT
GCACGATATAATCCTAGCAAATTGA

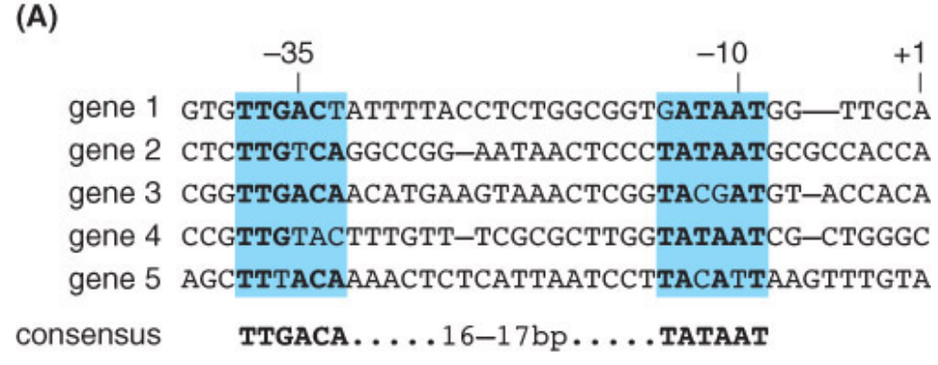
Motiv finding

-10 promoter consensus



Motiv finding

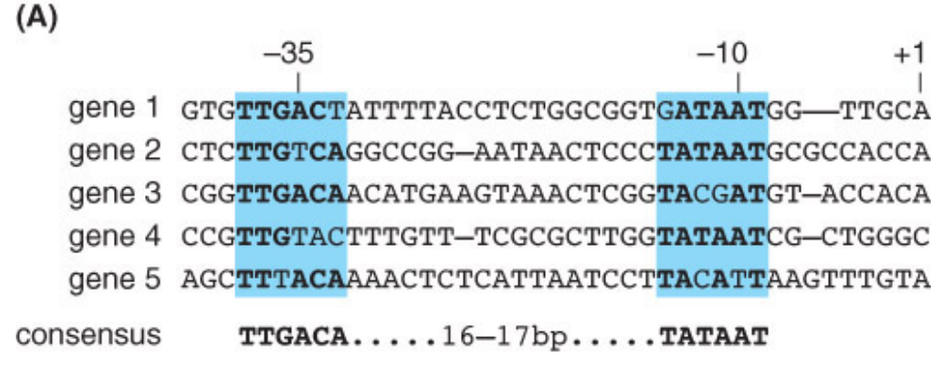
-10 promoter consensus



TATAAT
GCACGATATAATCCTAGCAAATTGA

Motiv finding

-10 promoter consensus

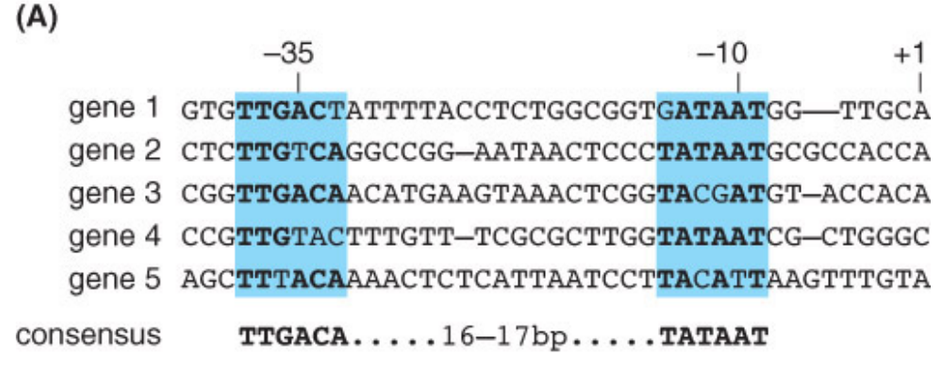


TATAAT

GCACGATATAATCCTAGCAAATTGA

Motiv finding

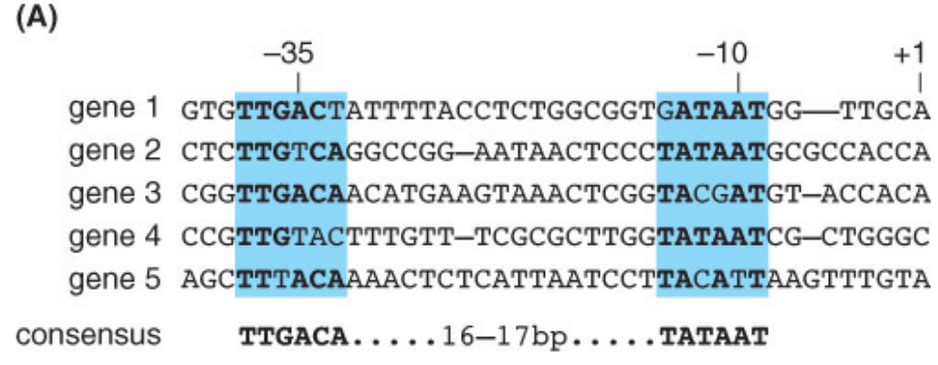
-10 promoter consensus



TATAAT
GCACGATATAATCCTAGCAAATTGA

Motiv finding

-10 promoter consensus



TATAAT

GCACGATATAATCCTAGCAAATTGA

OSV..

Konsensussekvenser

Table 9.3 Consensus sequences for gene expression in prokaryotes and eukaryotes.

Sequence	Consensus (5′ → 3′)	Function
Prokaryotes		
−10 sequence	TATAAT	RNA polymerase binds to start transcription
−35 sequence	TTGACA 17±2 from −10	RNA polymerase binds to start transcription
Shine-Dalgarno	AGGAGG 5±2 from ATG	Ribosome binds to find start codon
Eukaryotes		
TATA box	TATAWAW	Core promoter; binds TFIID
Inr sequence	YYCARR	Core promoter; contains +1 sequence (C)
GC box	GGGCGG	Transcription factor binding site
CAT box	CAAT	Transcription factor binding site
Kozak consensus	gccRccATGG	Context of start codon
5′ splice site	MAG | GTragt	Bound by spliceosome to remove introns
3′ splice site	cAG | G	Bound by spliceosome to remove introns
intron branch site	CTRAY	3′ end of intron binds to mark for degradation
polyadenylation site	AAUAAA	Cleavage of mRNA for poly(A) tail

```
M   = A, C
R   = A, G
W   = A, T
S   = C, G
Y   = C, T
K   = G, T

V   = A, C, G
H   = A, C, T
D   = A, G, T
B   = C, G, T

X   = G, A, T, C
N   = G, A, T, C
```

International **U**nion of **P**ure and **A**pplied **C**hemistry

```
IUPAC = {'T': 'T',
         'A': 'A',
         'G': 'G',
         'C': 'C',
         'M': 'AC',
         'R': 'AG',
         'W': 'AT',
         'S': 'CG',
         'Y': 'CT',
         'K': 'GT',
         'V': 'ACG',
         'H': 'ACT',
         'D': 'AGT',
         'B': 'CGT',
         'X': 'GATC',
         'N': 'GATC' }
```

International **U**nion of **P**ure and **A**pplied **C**hemistry

```
IUPAC = {'T': 'T',
         'A': 'A',
         'G': 'G',
         'C': 'C',
         'N': 'GATC' }
```

Skriv lige den af.

```
IUPAC = {'T': 'T',
         'A': 'A',
         'G': 'G',
         'C': 'C',
         'M': 'AC',
         'R': 'AG',
         'W': 'AT',
         'S': 'CG',
         'Y': 'CT',
         'K': 'GT',
         'V': 'ACG',
         'H': 'ACT',
         'D': 'AGT',
         'B': 'CGT',
         'X': 'GATC',
         'N': 'GATC' }
```

Test om kort sekvens passer til et IUPAC motiv

```
motif:  GNTC

seq:    GATC
```

Opgave

Write a function: **`test_motif`** that takes two arguments:

1. A string, which represents a short DNA sequence.
2. A string of the same length, which represents a IUPAC consensus sequence.

The function must return:

- **`True`** if the DNA sequence matches the consensus sequence and **`False`** otherwise.

Løsning

Opgave

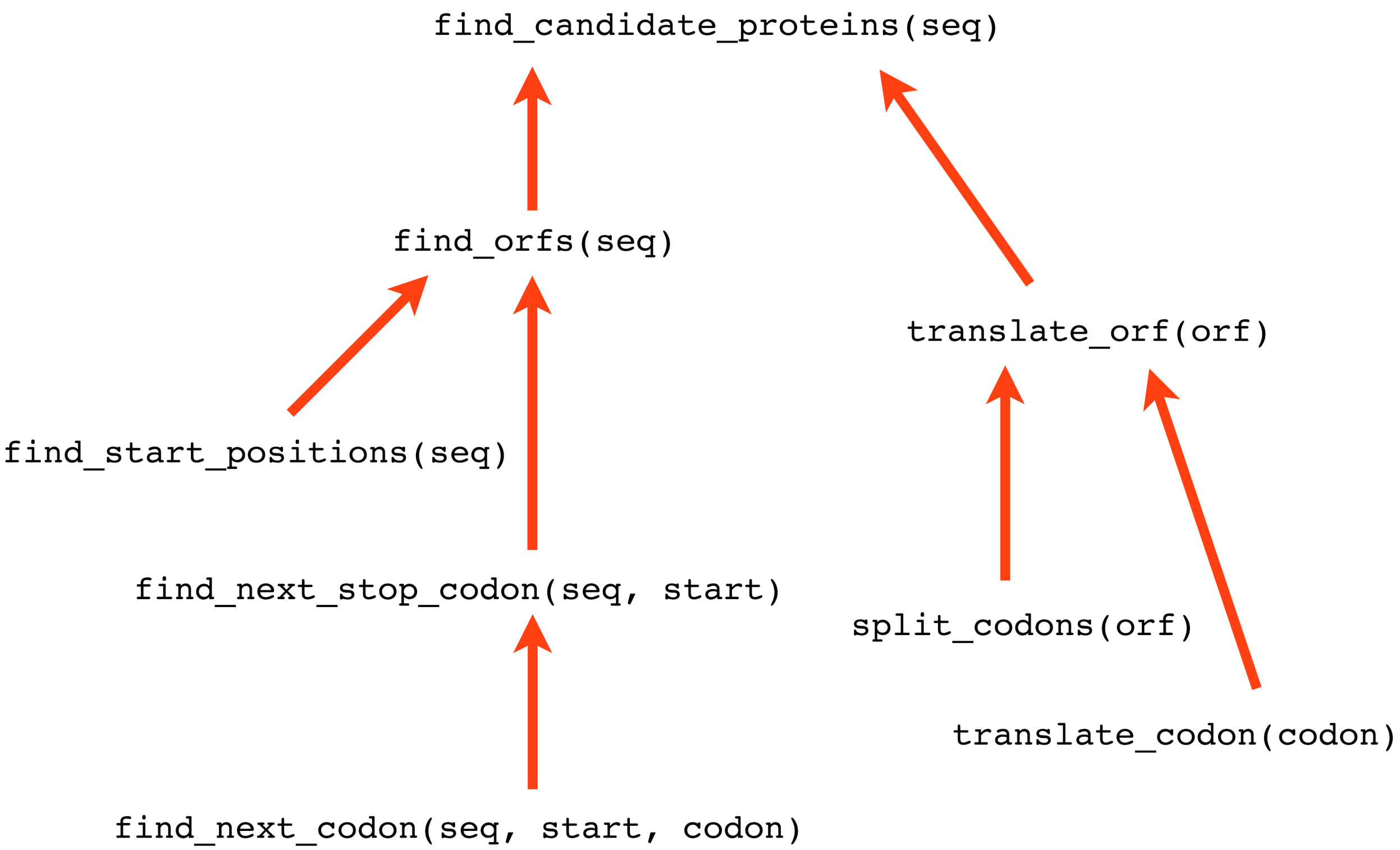
Write a function: **`find_motifs`** that takes two arguments:

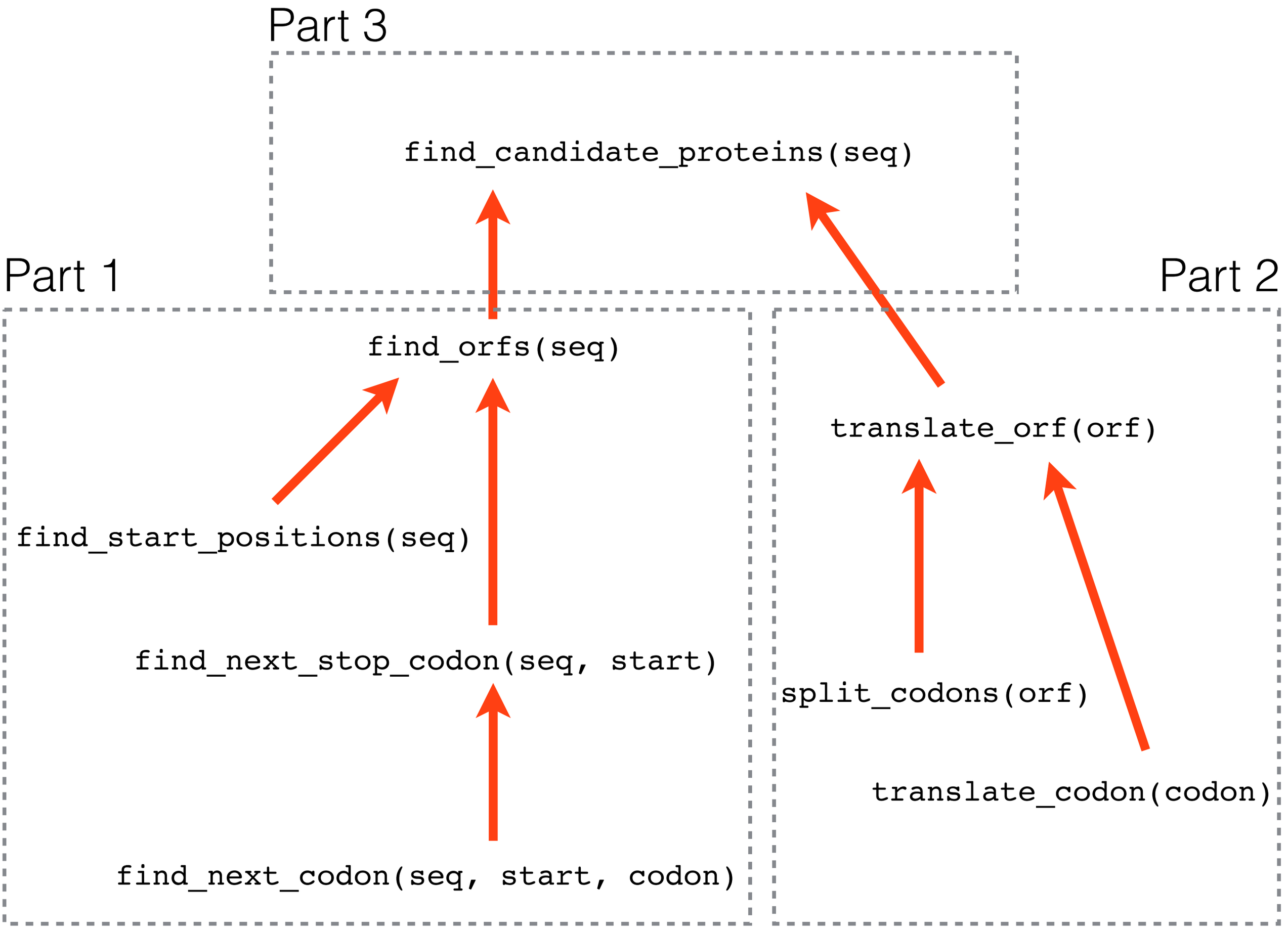
1. A string, which represents a long DNA sequence.
2. A string, which represents a IUPAC consensus sequence.

The function must return:

- A list of integers, which represent the indexes in the DNA sequence where a substring matching the consensus sequence starts.

Løsning


find_candidate_proteins(seq)
find_orfs(seq)
translate_orf(orf)
find_start_positions(seq)
find_next_stop_codon(seq, start)
split_codons(orf)
translate_codon(codon)
find_next_codon(seq, start, codon)


Part 3
find_candidate_proteins(seq)
Part 1
Part 2
find_orfs(seq)
translate_orf(orf)
find_start_positions(seq)
find_next_stop_codon(seq, start)
split_codons(orf)
translate_codon(codon)
find_next_codon(seq, start, codon)

Part 3

find_candidate_proteins(seq)

Translation project

Part 1

find_orfs(seq)

find_start_positions(seq)

find_next_stop_codon(seq, start)

find_next_codon(seq, start, codon)

Part 2

translate_orf(orf)

split_codons(orf)

translate_codon(codon)

- Put your **`translationproject.py`** in the same folder as your **`orfproject.py`**

- In **`orfproject.py`**:

```
from translationproject import (
    split_codons,
    translate_codon,
    translate_orf
)
```